Experiment No. 10

Aim: Create a Jenkins CICD Pipeline with SonarQube / GitLab Integration to perform a static analysis of the code to detect bugs, code smells, and security vulnerabilities on a sample Web / Java / Python application.

Objective: To train learners in performing Static Application Security Testing (SAST) using tools like SonarQube.

Outcome: CO3

Apply continuous software testing using SAST and Selenium, utilizing engineering tools to ensure code quality thereby foster lifelong learning to stay updated with evolving testing methodologies.

Theory :

 What is SAST?

Static Application Security Testing (SAST) is a frequently used Application Security (AppSec) tool, which scans an application's source, binary, or byte code. A white-box testing tool, it identifies the root cause of vulnerabilities and helps remediate the underlying security flaws. SAST solutions analyze an application from the "inside out" and do not reed a running system to perform a scan. Static application security testing (SAST), or static analysis, is a testing methodology that analyzes source code to find security vulnerabilities that make your organization's applications susceptible to attack. SAST scans anapplication before the code is compiled. It's also known as white box testing.

 What is SonarQube?

SonarQube is a Code Quality Assurance tool that collects and analyzes source code, and provides reports for the code quality of your project. It combines static and dynamic analysis tools and enables quality to be measured continually over time. Everything from minor styling choices, to design errors are inspected and evaluated by SonarQube. This provides users with a rich searchable history of the code to analyze where the code is messing up and determine whether or not it is styling issues, code defeats, code duplication, lack of test coverage, or excessively complex code. The software will analyze source code from different aspects and drills down the code layer by layer, moving module level down to the class level, with each level producing metric values and statistics that should reveal problematic areas in the source code that needs improvement.

Sonarqube also ensures code reliability, Application security, and reduces technical debt by making your code base clean and maintainable. Sonarqube also provides support for 27 different languages, including C, C++, Java, Javascript, PHP, GO, Python, and much more.SonarQube also provides Ci/CD integration, and gives feedback during code review with branch analysis and pull request decoration.

SonarQube is an open-source tool for continuous inspection of code quality. It performs static analysis of code, thus detecting bugs, code smells and security vulnerabilities. In addition, it also can report on the duplicate code, unit tests, code coverage and code complexities for multiple programming languages. Hence, in order to achieve Continuous Integration with fully automated code analysis, it is important to

integrate SonarQube with CI tools such as Jenkins.

 Benefits of SonarQube :

• Sustainability –

Reduces complexity, possible vulnerabilities, and code duplications, optimising the life of applications.

• Increase productivity –

Reduces the scale, cost of maintenance, and risk of the application; as such, it removes the need to spend more time changing the code

• Quality code –

Code quality control is an inseparable part of the process of software

development. • Detect Errors –

Detects errors in the code and alerts developers to fix them automatically before submitting them for output.

• Increase consistency –

Determines where the code criteria are breached and enhances the

quality • Business scaling –

No restriction on the number of projects to be evaluated

• Enhance developer skills –

Regular feedback on quality problems helps developers to improve their coding

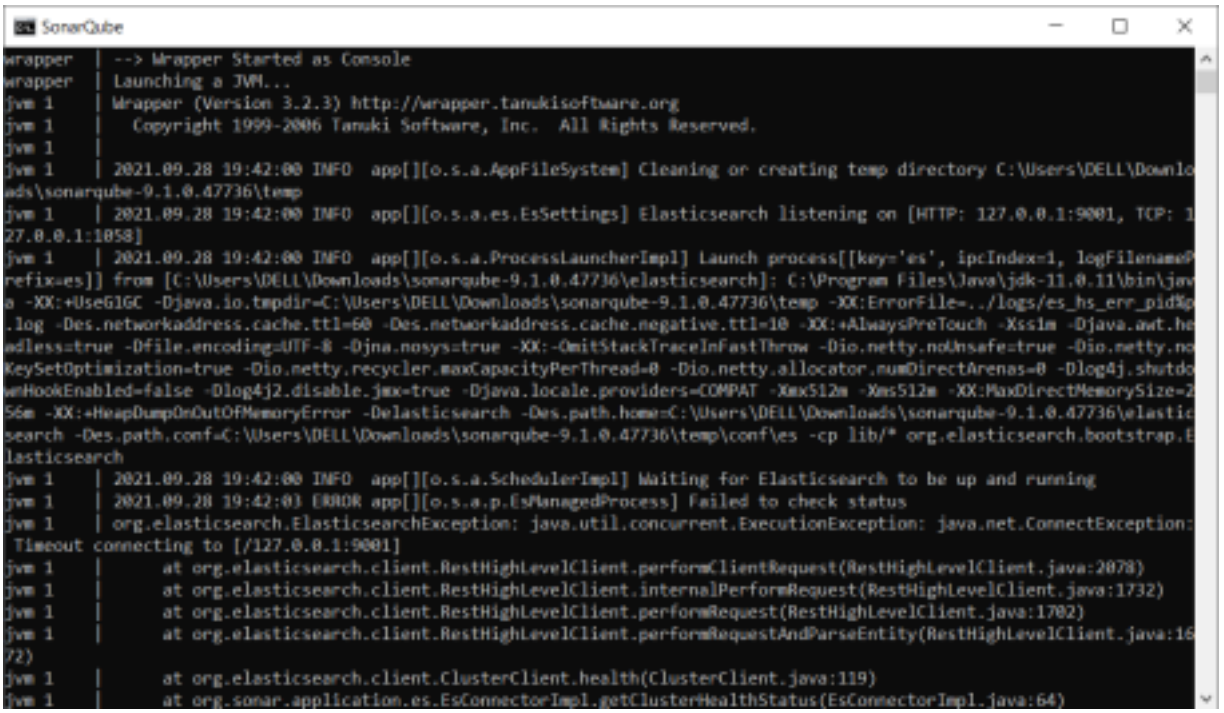
skills **Steps to install SonarQube on windows :**

Step 1. Download Community edition from https://www.sonarqube.org/downloads/

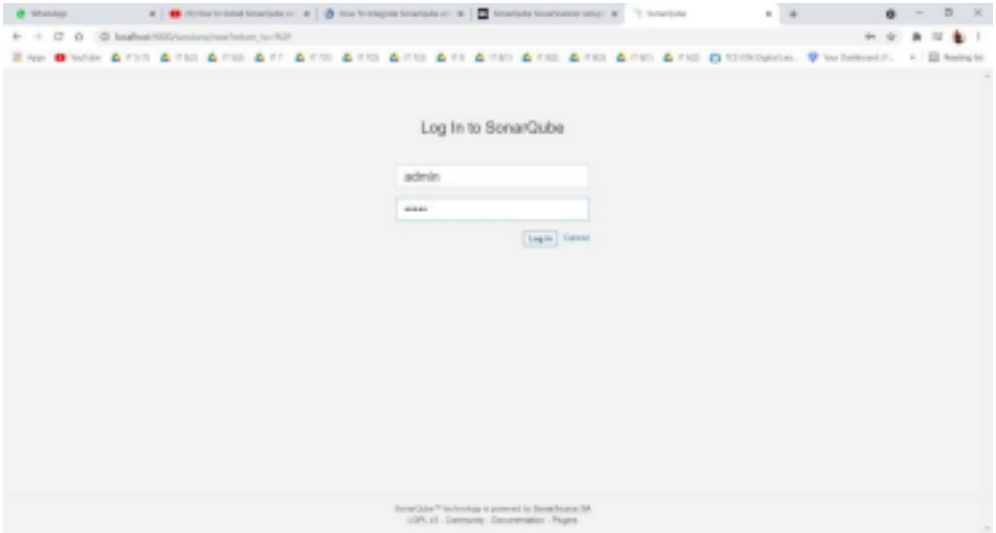
Step 2. Extract .zip file and navigate to bin folder

Step 3. Based on your machine configuration go to folder for 32-bit OS move to windows-x86– 32 and for 64-bit OS move to windows-x86–64

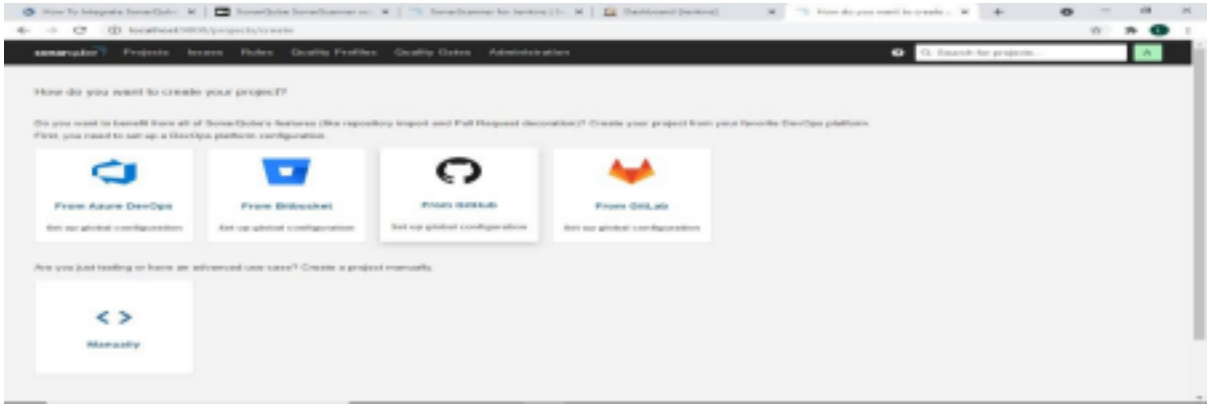
Step 4. Run StartSonar.bat and after few minute it will start your SonarQube server.



Step 5. Open browser and http://localhost:9000/ (9000 is default) you will be navigated to below window

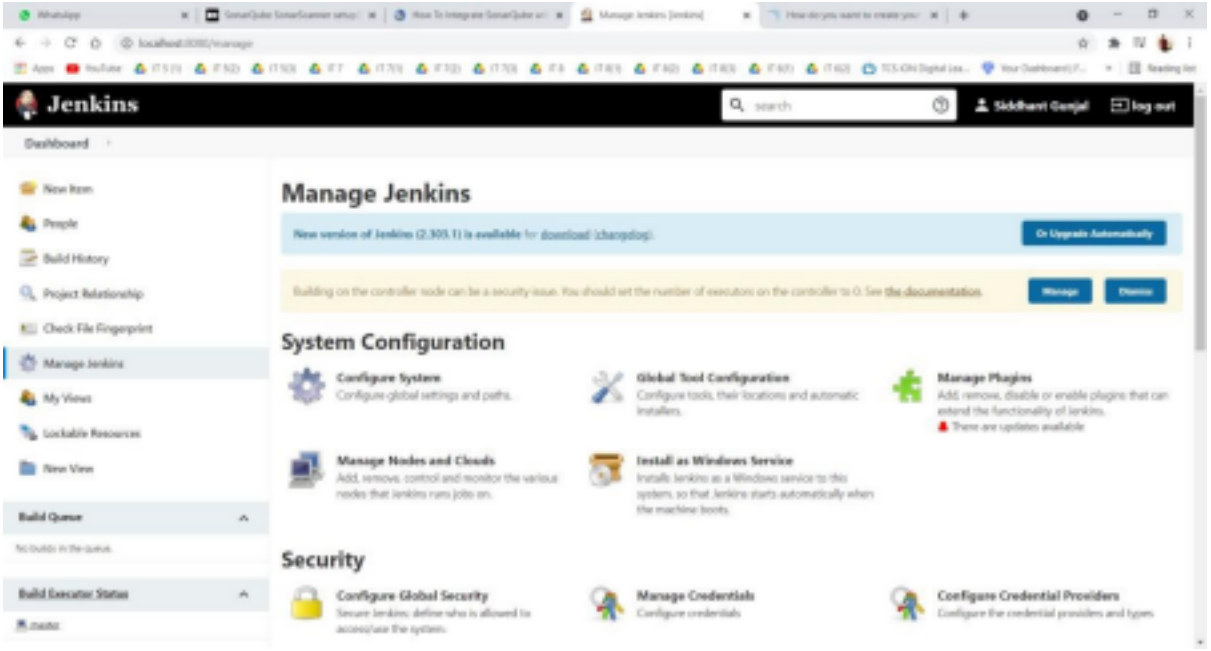


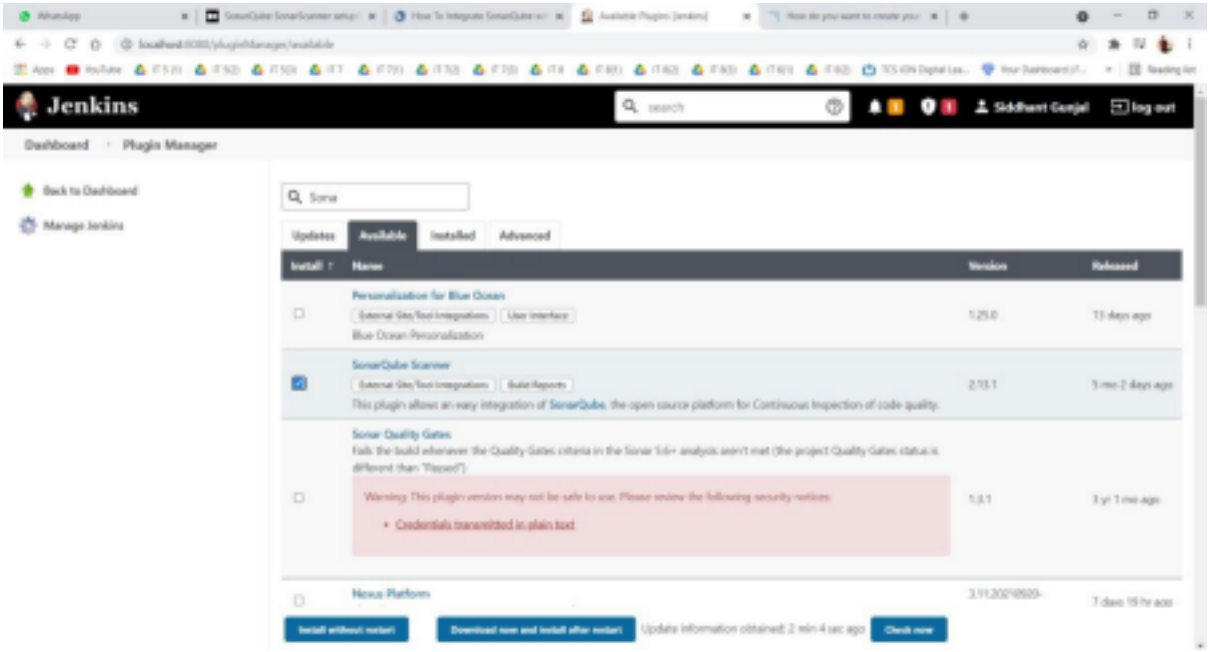
Default Login and Password is admin.

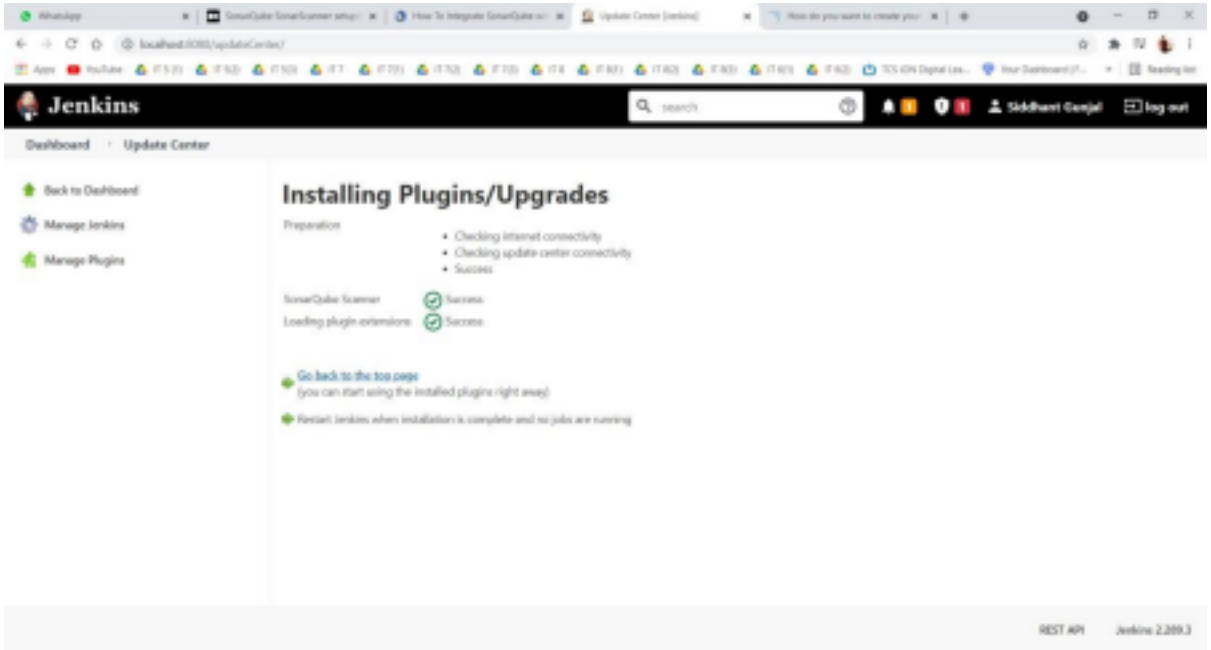


 Steps to integrate SonarQube with Jenkins.

Step 1 : The first step to integrate the sonarqube installation with jenkins devops environment is to generate an access token.

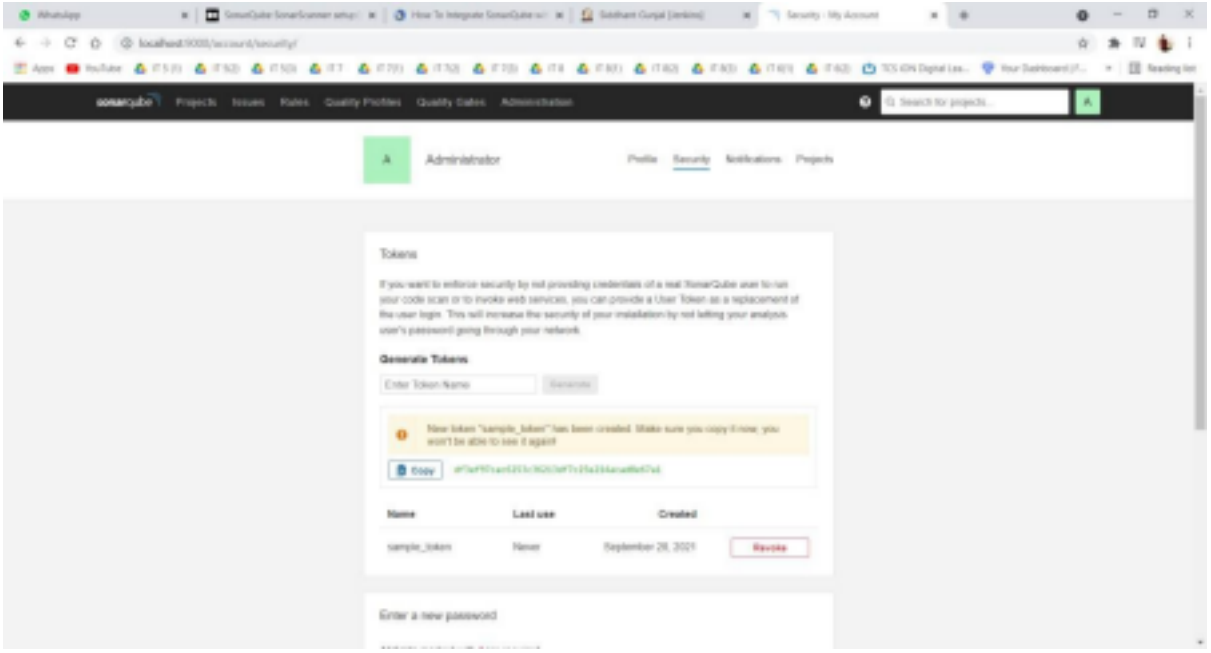






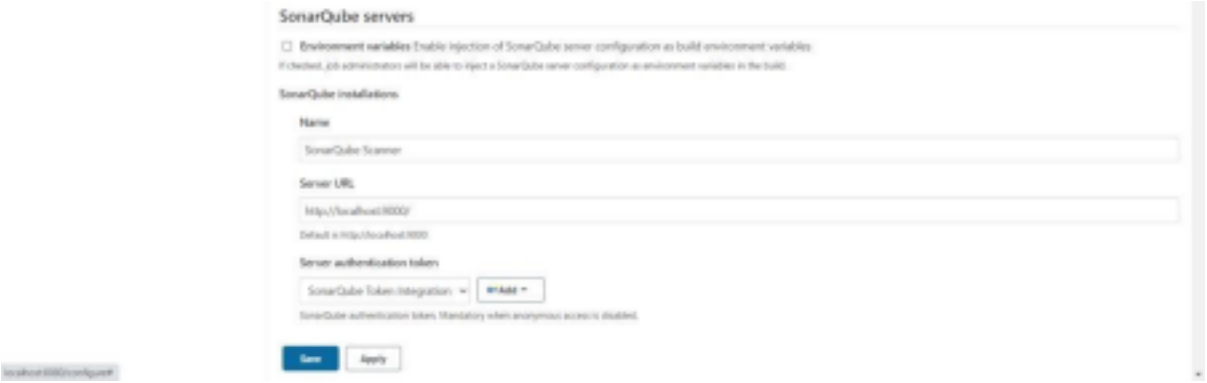
Go to Administration > Security > Users > Tokens > Generate token with some name > Copy the token

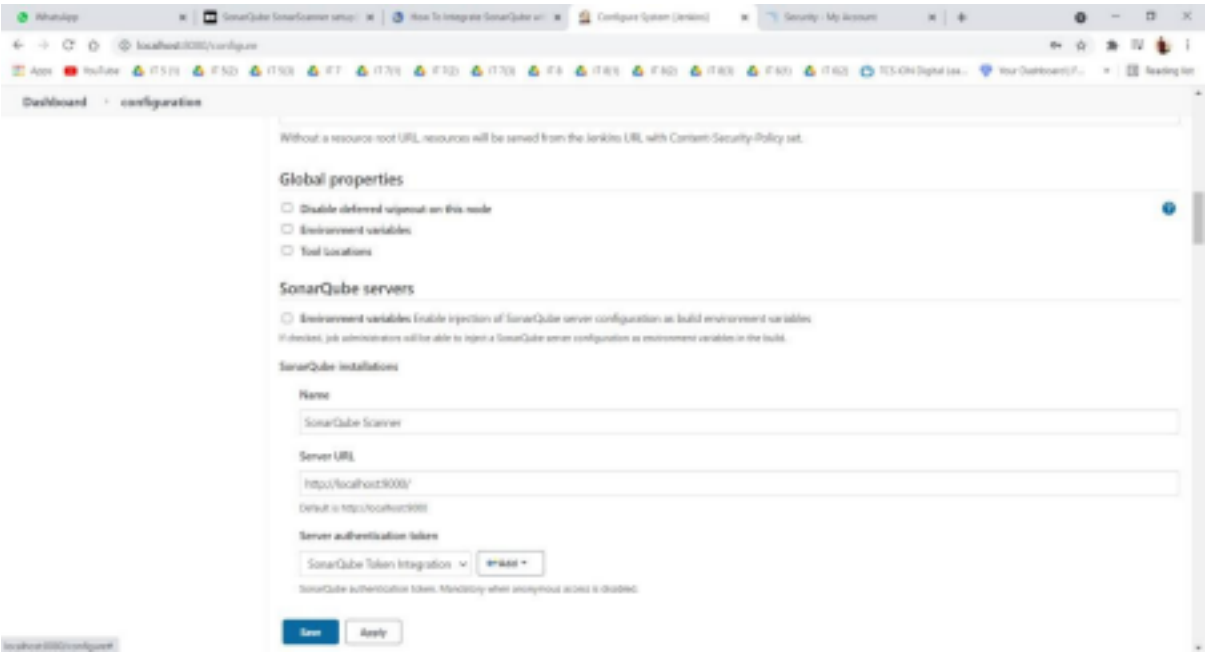
This token will be used in Jenkins for Sonar authentication



Step 2 : Then we need to configure the sonarqube installations with jenkins by using the generated access tokens. The steps to be followed in order to integrate with jenkins are,

Go to Manage Jenkins> Go to SonarQube servers section> Add SonarQube> Put a proper name(Your own choice)> Put the server URL as http://localhost:9000 if the server is running in the same server or if you install it as a separate server or if your running it on a different port you can put the respective Server URL> Click add> And select secret text> Add the generated token as secret key and save.

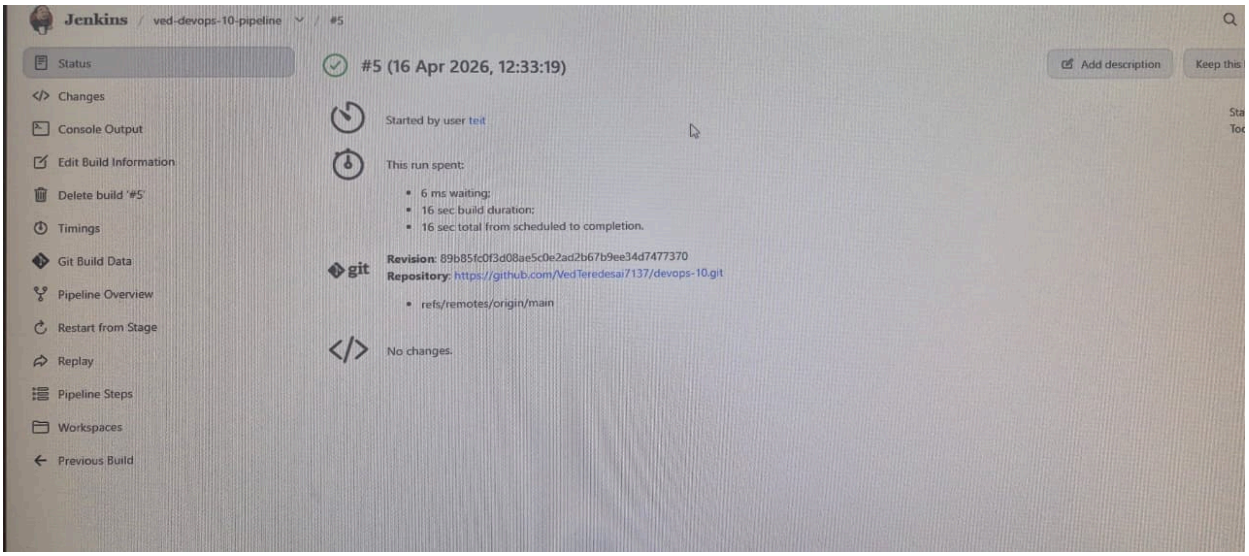




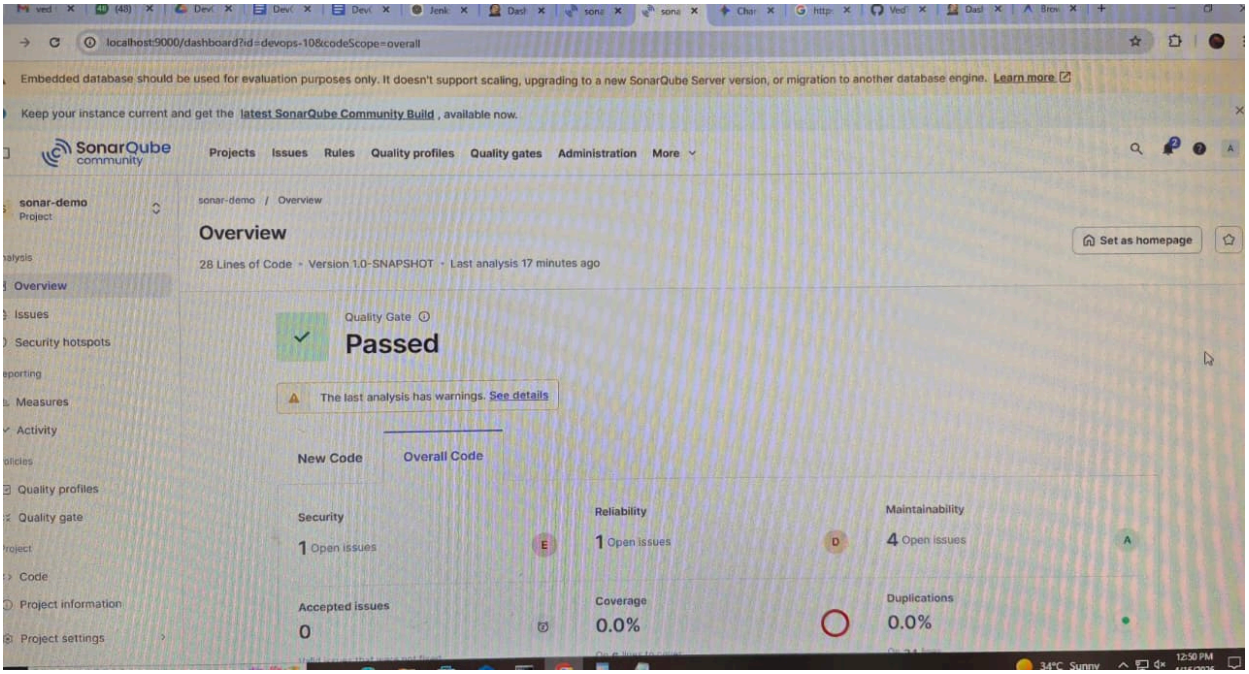
Step 3:- Make pipeline in jenkins and add build script

```
pipeline {
    agent any

    tools {
        maven 'Maven-3.9.12'
    }
```

```
    stages {

        stage('Checkout Code') {
            steps {
                git branch: 'main', url: 'https://github.com/VedTeredesai7137/devops-10.git'
            }
        }

        stage('Build') {
            steps {
                bat 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                bat 'mvn test'
            }
        }

        stage('SonarQube Analysis') {
            steps {
                withSonarQubeEnv('sonarqube') {
                    bat """
                    mvn sonar:sonar ^
                    -Dsonar.projectKey=devops-10 ^
                    -Dsonar.host.url=http://localhost:9000 ^
                    -Dsonar.login=squ_25a57563d9a0bbd10abd7a3dbe618e50984884d8
                    """
                }
            }
        }
    }
}
```



Step 4 :- Make changes and test correctly



Conclusion :

Hence, understood Static Analysis SAST process and successfully integrated Jenkins with SonarQube and performed static testing using SonarCube.